

# Top C++ STL Interview Questions and Answers

Last Updated : 23 Jul, 2025



The Standard Template Library (STL) is a set of C++ template classes that are used to implement widely popular algorithms and data structures such as vectors, lists, stacks, and queues. It is part of the C++ Language ISO standard. STL is a popular topic among interviewers, so it is useful for both freshers and experienced to learn the commonly asked interview question on STL in C++.

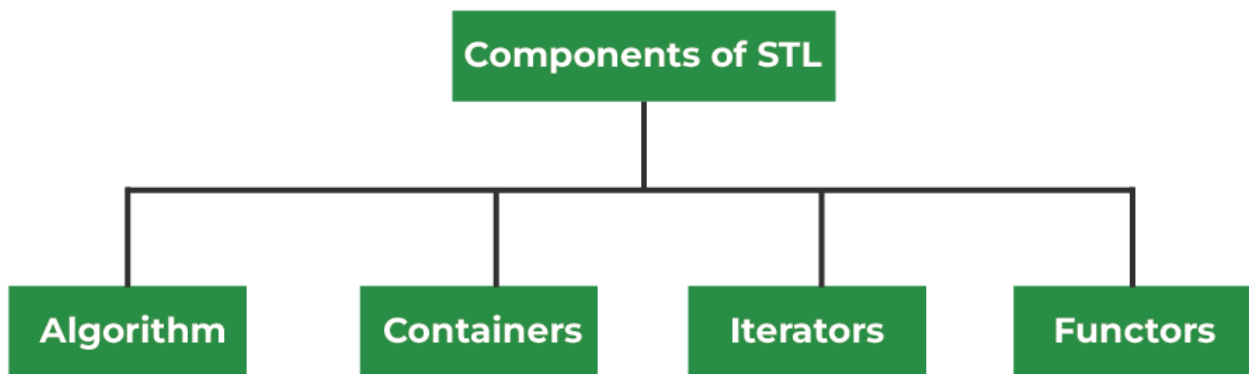


In this article, we will see the **top 50 most important and most frequently asked interview questions on C++ STL**.

## STL Interview Questions and Answers

### 1. What is STL?

STL stands for Standard template library and is the collection of some most commonly used algorithms and data structures such as vectors, maps, etc. It is a generalized library that works for all data types. STL has 4 components which are as follows:



For more information, refer to the article - [STL in C++](#).

## 2. What is a template?

C++ gives a feature that allows functions and classes to operate with generic types in the form of [templates](#). We only have to write the code of a function or a class once and it will be able to operate on different data types.

## 3. Why we use `<bits/stdc++.h>`?

The `<bits/stdc++.h>` is a header file that is used to include all the standard header files at once in the C++ program. It is mostly used in programming contests to save the time required for including header files one by one. But remember that it itself is a non-standard header file of the GNU C++ Library so not all compilers have it.

## 4. Why do we need STL when we can perform all the operations using a user-defined data structure and functions?

We prefer STL over user-defined data structure and functions because:

- It saves time spent writing code for user-defined data structures.
- It is [tried](#), efficient, and debugged code tested over a long period of time.
- STL is the part of C++ Language Standard so it is easy to understand for other programmers.
- STL allows us to parameterize the code with the required data type.

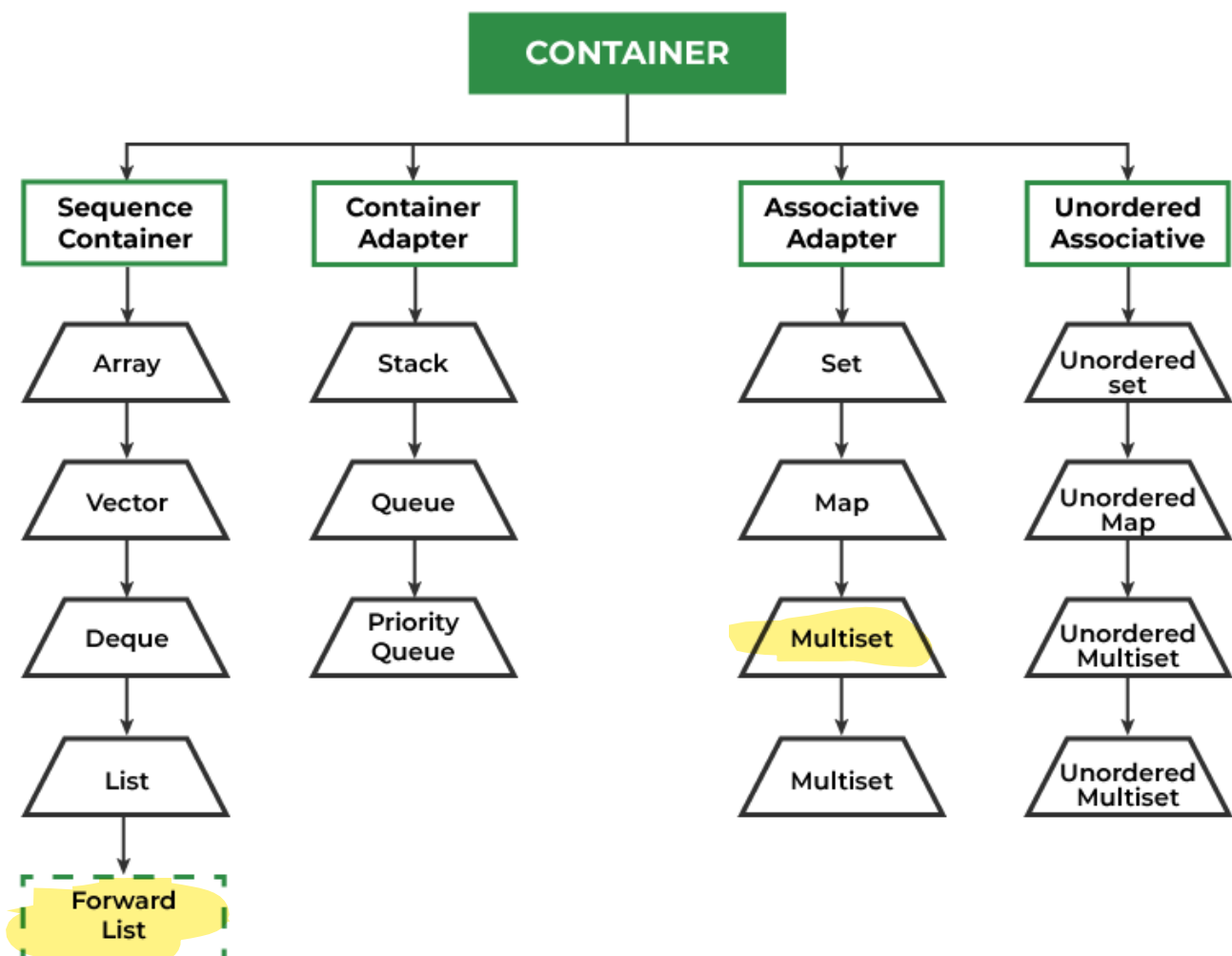
## 5. What are containers in STL?

The Containers in STL are class templates using which we implement data structures like a queue, stack, etc. These containers act similarly to their corresponding data structure, manage the storage space for its elements and provide member functions to access them.

For Example, a Vector acts similarly to a dynamic array and has methods like `push_back()`, `pop_back()`, `size()`, etc.

Containers are of three types:

- Sequence container
- Associative container
- Derived container



*Different Types of Containers in STL*

For more information, refer to the article - [Containers in STL](#).

## 6. What are Algorithms in STL?

Algorithms in STL is a library that contains some commonly used methods predefined as function templates. They generally work on containers taking their iterators as arguments. They are defined inside the `<algorithm>` header file.

## 7. What are **Functors in STL**?

A functor (**or function object**) is a C++ class that acts like a function. Functors are called using the same old function called syntax. To create a functor, we overload the operator( ).

**Example:**

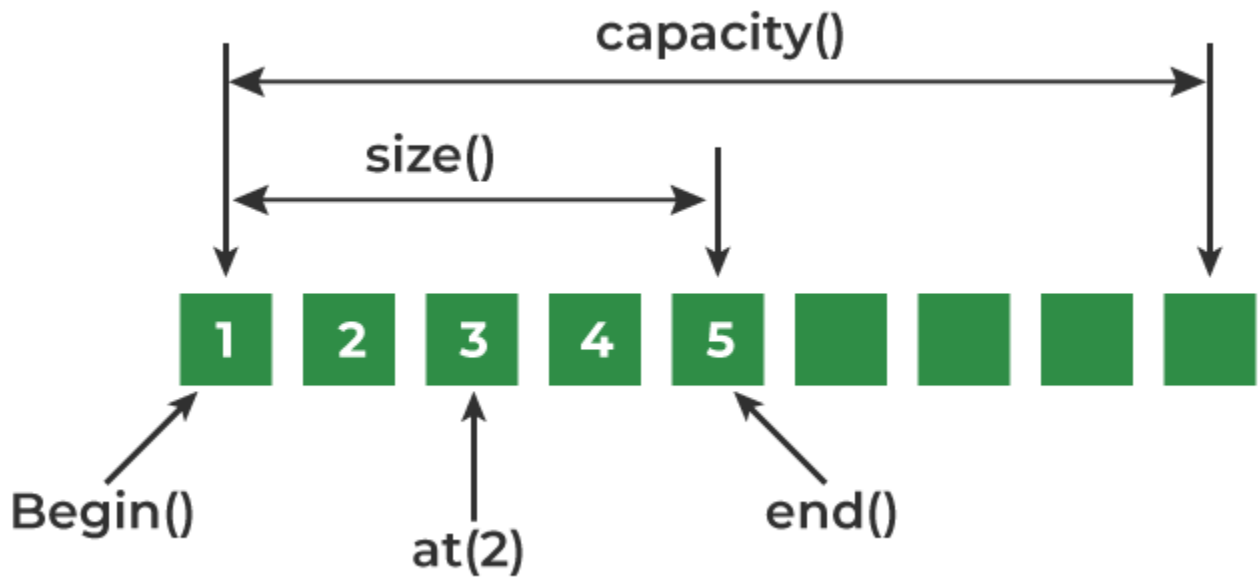
```
MyFunctor(10);  
MyFunctor.operator( )(10);           //Both act same
```

## 8. What is a vector?

A vector is a type of container that **holds the same properties as** a dynamic array. It is a sequential container in which we can randomly access any element using an index number but can only insert or delete elements from the end in constant time using `push_back( )` and `pop_back( )` respectively.

**Syntax:**

```
vector<object_name> vector_name;
```



*Vector in C++*

## 9. What is an iterator?

An iterator is **a variable** that points to an element in an STL container and can be used to traverse through the elements in the container.

### Syntax:

```
vector<int>::iterator itr;
```

Here `itr` is the iterator that can be used for iteration over a `vector<int>`.

## 10. What is the **range** in terms of vectors?

A range is a sequence of consecutive elements in a container. The range can be specified from `begin()` to `end()`, containing all the elements of a container.



## 11. What is the difference between an array and a vector?

The differences between an array and a vector in C++ are as follows:

| Array                                                 | Vector                                                               |
|-------------------------------------------------------|----------------------------------------------------------------------|
| The array is a type of data structure.                | Vector is a type of Container                                        |
| Its size is fixed after declaration.                  | Vector is dynamically resizable that can change its size when needed |
| Elements of an array are stored in stack memory.      | Elements of the vector are stored in the free store.                 |
| The size can be obtained by traversing.               | Size can be easily determined using the size() member function.      |
| <b>Syntax:</b> <code>data_type arr_name[size];</code> | <b>Syntax:</b> <code>vector&lt; object_type &gt; name;</code>        |
| <b>Example:</b> <code>int arr[5];</code>              | <b>Example:</b> <code>vector&lt;int&gt; arr;</code>                  |

## 12. How can we insert elements in a vector?

We can insert elements using 4 methods:

- Using `push_back()` function
- Using `insert()` function
- Using `emplace()` function
- Using `[]` (array subscript)

### A. Using `push_back()`:

This method is used when we want to insert an element at the last position. The size of the vector will be increased using `push_back()`. It is the most used method of insertion in vectors.

**Example:**

```
vect.push_back(12); //here arr is int vector and 12 is value inserted.
```

### B. Using `insert()` function

We can also use the `insert()` member function to insert elements in a vector at some particular position.

**Example:**

```
vect.insert(vect.begin(), 20);
```

This will insert the 20 at the index 0 of the vector.

### C. Using `emplace()` function

We can also use the `emplace()` member function to insert elements in a vector at some particular position in a similar way to `insert()` functions.

**Example:**

```
vect.emplace(vect.begin(), 20);
```

This will insert the 20 at the index 0 of the vector.

### D. Using `[]` (Array Subscript):

If the size of the vector is predeclared, then we can directly insert elements using [ ] operators.

**Example:**

```
vect[i] =12;           // value of ith index will be 12 now.
```

### 13. How can we remove elements in a vector?

We can remove elements in vectors using two methods:

- Using pop\_back() function
- Using erase() function

#### A. Using pop\_back() function

pop\_back() is a member function of the vector class and it is used to remove the last element from the vector.

**Example:**

```
vect.pop_back();      // last element will be removed.
```

#### B. Using erase() function

erase() is also a member function of the vector class. It is used to remove the elements at a particular position in the vector.

**Example:**

```
vect.erase(vect.begin() + 2);  // last element will be removed.
```

### 14. What is the time complexity of insertion and deletion in vector?

**Insertion:** If the size of the vector is N, then the time complexity of insertion of an element in the vector is:

- At the end: **O(1)**
- At M index: **O(N - M)**

**Deletion:** If the size of the vector is N, then the time complexity of deletion of an element in the vector is:

- At the end:  **$O(1)$**
- At M index:  **$O(N-M)$**

## 15. What is the use of auto keyword in C++?

The **auto** keyword specifies that the type of variable that is being declared will be automatically deducted from its initializer. In the case of functions, if their return type is auto then that will be evaluated by the return type expression at runtime. Good use of auto is to avoid long initializations when creating iterators for containers.

## 16. How can we traverse a vector?

We can traverse in a vector with the following methods:

- Using Index
- Using an Iterator
- Using auto Keyword

### i) Using index

We can access the elements of the vector using the index number in a similar way to the arrays.

```
for (int i = 0; i < v1.size(); i++)
{
    cout << v1[i] << " ";
}
```

### ii) Using an iterator

Iterators are the objects just like pointers that point to the elements inside the containers. Iterators are used to iterate over the elements of the containers.

```
vector<int>::iterator itr;

for (itr = v1.begin(); itr < v1.end(); itr++)
{
    cout << *itr << " ";
}
```

### iii) Using auto

The auto keyword specifies the type of variable that is being declared will be automatically deduced from its initializer.

```
for (auto it:v)
{
    cout << it << " ";
}
```

## 17. How to print vectors in C++?

We can print vectors using multiple ways:

- Using overloading << Operator
- Comma separated manner
- Using indexing
- One line without for loop
- Using (experimental::make\_ostream\_joiner) without providing element type
- One line using the lambda function

For more information, refer to the article - [Print vector in C++](#).

## 18. How can we convert the array into a vector?

There are a few methods to convert array into a vector:

- While index iteration of array push elements
- Range-based Assignment during Initialization
- Using Inbuilt Function Insert( )

- Using Inbuilt Function Copy( )
- Using Inbuilt Function Assign( )
- Using Inbuilt Function Transform( )

For more information, refer to the article - [Convert the array into a vector.](#)

## 19. How can we convert vectors into arrays?

We can convert the vector into an array using multiple ways:

- By copying items one by one
- Using STL Algorithm copy()
- Using STL Algorithm transform()
- Using vector::data()

## 20. How to initialize a 2-D vector in C++?

2-D vector can be initialized using multiple methods:

**Syntax:**

```
vector < vector<object_type> > vector_name;
```

There are some instances like:

**i)** If we want to insert elements during initialization

```
//v vector containing these values
vector<vector<int>> v={{1,2,3},{4,5,6},{7,8,9}};
```

**ii)** If the number of rows is given:

```
//rows is the number of rows
vector<vector<int>> v(rows);
```

**iii)** If the number of rows and columns both are given:

```
//rows is the number of rows
//cols is the number of columns
```

```
vector<vector<int>> v(rows, vector<int> (cols));
```

iv) If all the values of the vector should be initialized by x

```
//rows is the number of rows  
//cols is the number of columns  
vector<vector<int>> v(rows,vector<int> (cols,x));
```

## 21. What is the time complexity of the sorting done in vector using the `sort( )` function?

STL provides the `sort( )` function, which operates with the best time complexity possible that can be obtained out of every sorting algorithm. So, the time complexity is  **$O(N \log N)$** .

## 22. What is the use of `lower_bound()` and `upper_bound()`?

STL algorithms provide the functionality to find lower and upper bounds.

- A lower bound of x is the number whose value is at least x.
- An upper bound of x is the number that comes just after x.

These functions are only effective when the vector is sorted because they use the binary search algorithm.

**Example:**

```
vector<int> a={2,3,4,5,6,7,8,8,10};  
  
auto x=lower_bound(a.begin(),a.end(),5);  
auto y=upper_bound(a.begin(),a.end(),5);
```

**Output:**

```
*x=5 , *y=6
```

## 23. What is a pair in STL?

Pair is a type of container that can store two elements of the same or different data types.

### Syntax:

```
pair<data_type,data_type> pair_name;
```

Elements of pair can be accessed by using first and second keywords. The first keyword is used to access the first element and the second keyword is used for accessing the second element.

## 24. Explain different methods to insert elements in a pair.

We can insert elements in a pair using three ways:

1. Directly inserting using the first and second keywords
2. Using make\_pair() function
3. Using { } braces

Consider a pair:

```
pair<int,string> x;
```

### i). Using first and second keywords

```
x.first = 1 ;  
x.second= "Geeks";
```

### ii). Using make\_pair( )

```
x=make_pair(1,"Geeks");
```

### iii). Using curly brackets

```
x={1, "Geeks"};
```

## 25. In which header file is the std::pair defined?

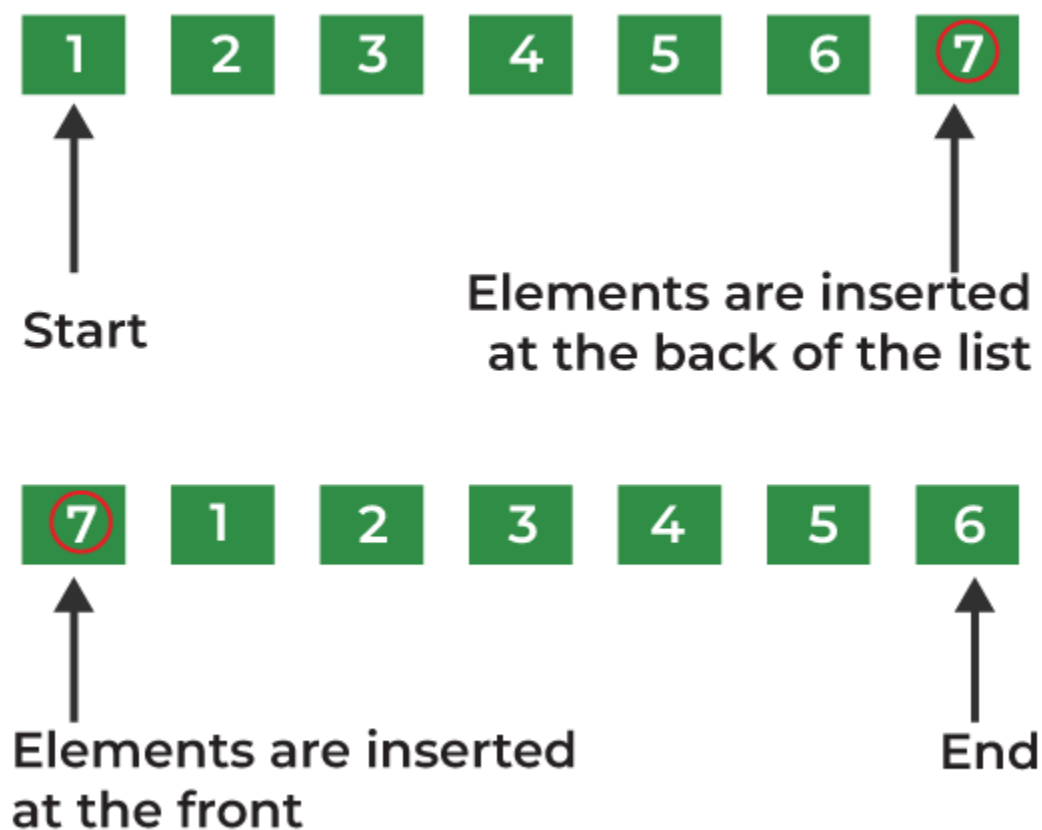
The `std::pair` class is defined inside the `<utility>` header file.

## 26. What is a List?

A list is a type of container in C++ that implements the doubly linked list data structure. The list provides non-contiguous storage with only sequential element access i.e we can't randomly access any element using an index number.

### Syntax:

```
list <object_name> list_name;
```



*List in C++*

## 27. What is the time complexity of insertion and deletion in the list?

**Insertion:** Suppose the size of the list is  $N$ , then the time complexity of insertion of an element in the list is:

- **At the end:**  $O(1)$
- **At the beginning:**  $O(1)$
- **At M index:**  $O(M)$

**Deletion:** Suppose the size of the list is N, then the time complexity of deletion of an element in the list is:

- **At the end:**  $O(1)$
- **At the beginning:**  $O(1)$
- **At M index:**  $O(M)$

## 28. Difference between a vector and a list?

| Vector                                                                         | List                                                                                |
|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Insertion at the end requires constant time but insertion elsewhere is costly. | Insertion is cheap no matter where in the list it occurs once the element is found. |
| Random access to elements is possible.                                         | Random access to elements is not possible.                                          |
| It has contiguous memory.                                                      | While it has non-contiguous memory.                                                 |
| A vector may have a default size.                                              | The list does not have a default size.                                              |
| Iterators become invalid if elements are added to or removed from the vector.  | Iterators are valid if elements are added to or removed from the list.              |
| <b>Syntax:</b> <code>vector&lt;data_type&gt; vector_name;</code>               | <b>Syntax:</b> <code>list&lt;data_type&gt; list_name;</code>                        |

For more information, refer to the article - [Vector vs List](#).

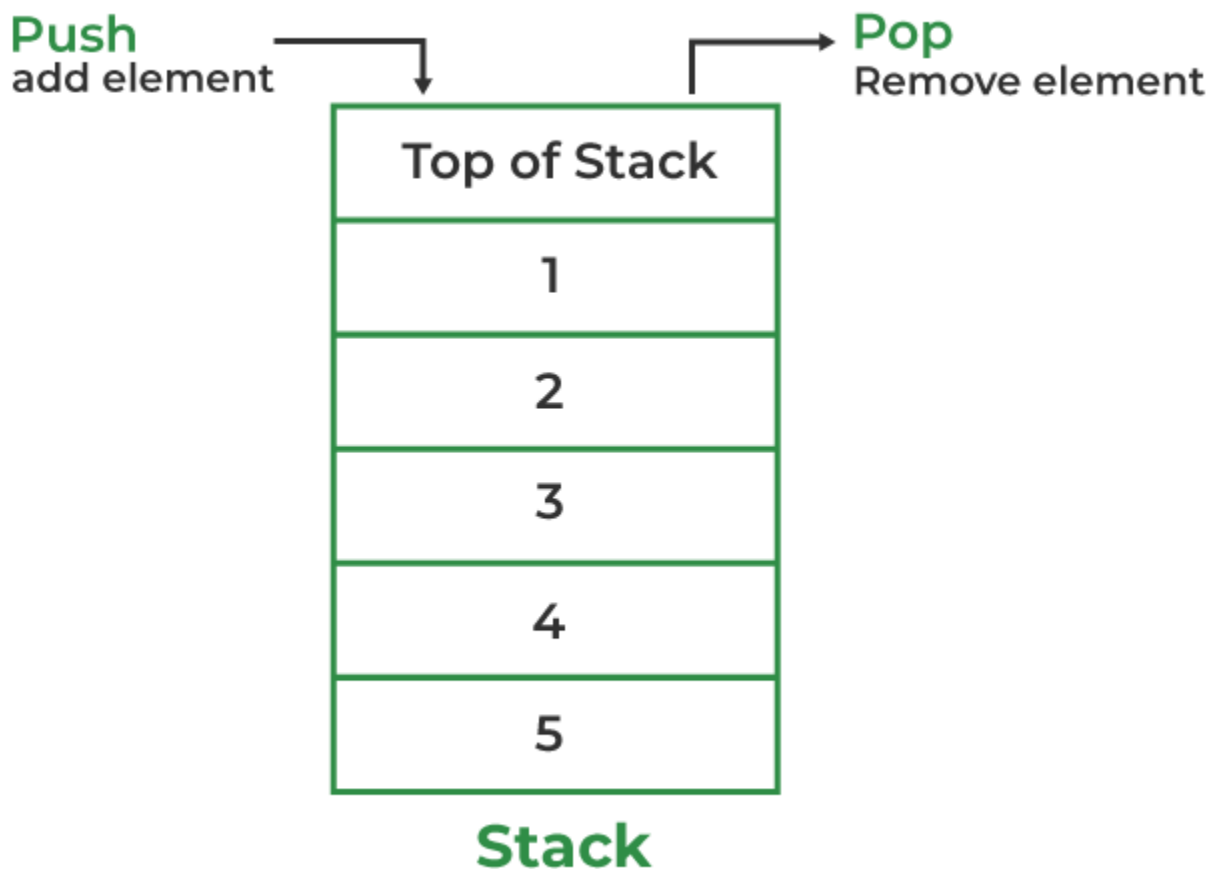
## 29. How can we remove elements from the list?

There are a few methods to remove elements from the list:

- Using `list::erase()`
- Using `list::pop_front()` and `list::pop_back()`
- Using `remove()` and `remove_if()`

For more information, refer to the article - [Remove elements from the list.](#)

### 30. What is a stack?



A stack is a container adapter that implements the stack data structure in STL. It has the following properties:

1. Elements follow the LIFO (Last In First Out) order of operation.
2. Only the element at the top can be accessed.
3. Insertion and Removal can be done only from the top.

#### Syntax:

```
stack <data_type> name;
```

Example: Chairs put one above the other so we can now either put, remove or access the chair at the top.

### 31. What are the basic functions associated with the STL stack?

The basic functions associated with the STL stack are as follows:

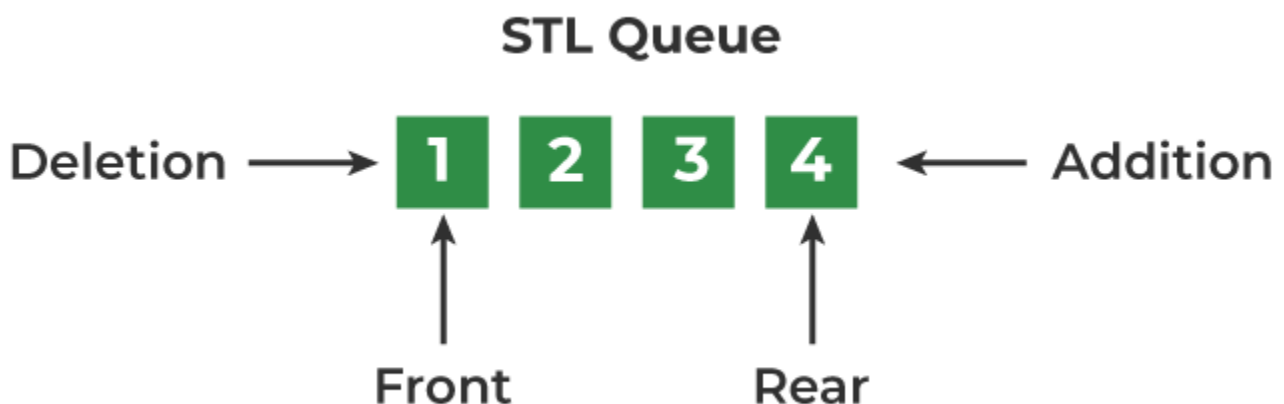
- **push()**: This function is used for inserting elements at the top of the stack.
- **pop()**: The pop() function is used for removing elements from the top.
- **size()**: It returns the size of the stack.
- **top()**: The top() function returns the top element of the stack.
- **empty()**: It checks if the stack is empty or not.

### 32. What is the time complexity of insertion and deletion in the stack?

Insertion and Deletion are only possible at the end of the container with the time complexity of **O(1)**. At any other position if you want to insert an element then we need to use another container or can use another stack, copy the element and remove it from the main stack then push the element at that position after then we need to push all elements again.

So, adding or removing at the M position we need to remove N-M elements and then add them again so,  $O(2*(N-M))$  approx **O(N)**.

### 33. What is a queue in STL?



A queue in C++ STL is a container adapter that implements the queue data structure. It has the following properties:

- Follows FIFO ( First In First Out) order of operation
- Queue allows insertion from one side and removal from another side.
- Insertion and removal both take  $O(1)$  time complexity.
- Only the element at the front is accessible.

#### Syntax:

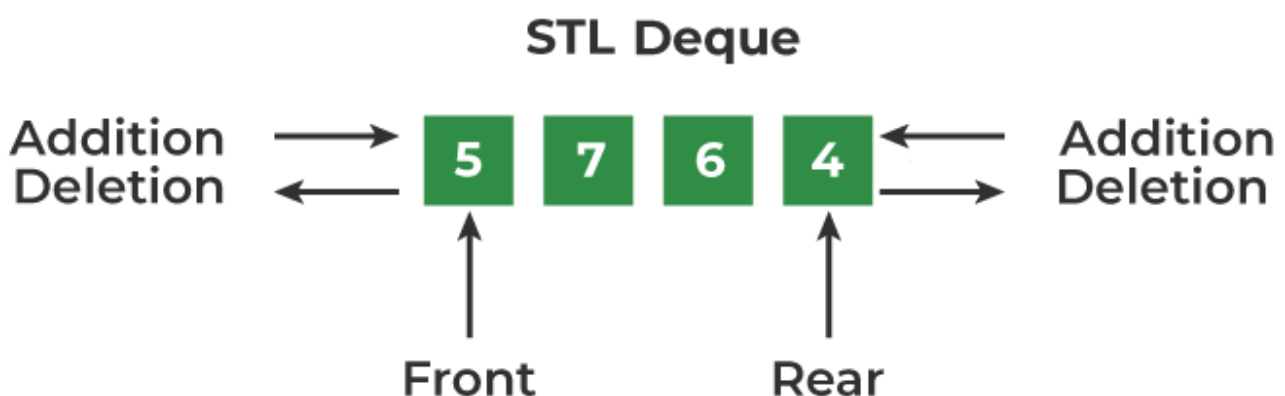
```
queue <data_type> name;
```

### 34. What are the commonly used member functions of the STL Queue?

Some commonly used functions associated with queue:

- **push( x )**: It is used to insert **x** in the queue.
- **pop( )**: Removes the least recent element from the list i.e. element at the front.
- **front( )**: It is used for accessing the front element.
- **size( )**: This function returns the size of the queue.

### 35. What is a deque?



Deque is also known as a Double-ended queue. Deque is a type of container that can insert and remove elements from both ends. It can push using `push_back( )` and `push_front( )` for inserting elements from the back and front respectively and

can remove using `pop_back( )` and `pop_front( )` for removing elements from the back and front respectively.

**Syntax:**

```
deque <object_type> deque_name;
```

### 36. What is the time complexity of insertion and deletion in the deque?

Insertion and Removal of elements are possible from both sides of the deque, both insertion and deletion take  $O(1)$  time complexity from either side.

For inserting elements at index  $M$  somewhere inside the deque, the time complexity will be linear i.e  $O(N)$ .

### 37. What is Set & How can we change the sorting order of a set?

A set is a type of associative container which stores value in a sorted way without duplication. It is sorted in increasing order by default.

**Syntax:**

```
set <object_type> name;
```

The set is implemented on a *Binary Search Tree (generally red-black tree)*, because of which time complexity of the elements that are stored in sorted form has the complexity of insertion, find and removal is  $O(\log N)$

For more information, refer to the article - [Set in C++](#).

In the set, all the elements stored are sorted in increasing order, but we can change the sorting order of the set to decreasing by using a `greater<int>` (STL functor) as a comparator in the set declaration.

**Syntax:**

```
set <data_type, greater<data_type>> name;
```

Similarly, we can use any comparator function, functors, or lambda expressions in place of `greater<int>` for custom sorting order.

### 38. How to access elements in a set by index?

We can access the element at the Nth index in a set by:

- Using Iterator
- Using `std::advance()` method
- Using `std::next()` method

To know more about these methods, please refer to this article - [How to Access Elements in Set by Index in C++?](#)

### 39. How to iterate over the set?

We can iterate over the STL set using the following methods:

- Iterate over a set using an iterator.
- Iterate over a set in a backward direction using `reverse_iterator`.
- Iterate over a set using a range-based for loop.
- Iterate over a set using `for_each` loop.

For more information, refer to the article - [Iterate over the set.](#)

### 40. What is a multiset in STL?

A multiset is an associative container that acts the same way as a set but allows duplicate values.

**Syntax:**

```
multiset<object_type> name;
```

The multiset is also implemented using *Binary Search Tree*. The time complexity of the elements stored in sorted form has the complexity of insertion, find and removal is  $O(\log N)$ .

For more information, refer to the article - [Multiset in C++](#).

## 41. What is a unordered\_set?

The unordered\_set is an unordered associative container that stores values in an unsorted way without duplication.

### Syntax:

```
unordered_set <object_type> name;
```

The unordered\_set is implemented using the *Hash table data structure*. The time complexity of the elements stored in sorted form has the complexity of insertion, find and removal is  $O(1)$  for average cases and  $O(n)$  time in the worst case.

For more information, refer to the article - [unordered\\_set in C++](#).

## 42. What is a unordered\_multiset in STL?

The unordered\_multiset is an unordered associative container that stores values in unsorted way and allows duplicate values.

### Syntax:

```
unordered_multiset <object_type> name;
```

The unordered\_multiset is based on the Hash-table data structure. The time complexity of the elements stored in sorted form has the complexity of insertion, find and removal is  $O(1)$  for average cases and  $O(n)$  for the worst case.

For more information, refer to the article - [unordered\\_multiset in C++](#).

## 43. What is a map?

The map in STL is an associative container that stores the data in the form of key-value pairs. It is sorted according to keys and each key is unique.

### Syntax:

```
map <object_type> name;
```

The map is generally **implemented on the Red-Black Tree (Self Balancing B.S.T.) data structure**. The time complexity for search, insert and delete operations is  $O(\log N)$ , where  $N$  is the number of key-value pairs in the map.

For more information, refer to the article - [Map in C++](#).

#### 44. What is a multimap?

A Multimap is a type of associative container that is similar to a map i.e storing key-value pairs and sorting according to keys, but the difference is that there can be multiple values associated with a single key.

##### Syntax:

```
multimap <object_type> name;
```

The multimap is also based on the *Balanced Binary Tree data structure*. The time complexity of the search, insert, and delete operations is logarithmic i.e.  $O(\log N)$ .

For more information, refer to the article - [Multimap in C++](#).

#### 45. What is a unordered\_map?

An unordered\_map is a type of unordered associative container that is similar to a map but the values are not sorted in any order.

##### Syntax:

```
unordered_map <object_type> name;
```

The unordered\_map is based on the *Hash Table*. The time complexity of the insert, delete, and search operations is  $O(1)$  for average cases and  $O(N)$  for the worst case.

For more information, refer to the article - [unordered\\_map in C++](#).

#### 46. What is a unordered\_multimap?

An `unordered_multimap` is also an unordered container that stores key-value pairs in an unsorted way. We can map multiple values to the same key in this container.

**Syntax:**

```
unordered_multimap <object_type> name;
```

The `unordered_multimap` is based on the *Hash Table*. The time complexity of the elements that are stored in sorted form has the complexity of insertion, find and removal is  $O(1)$  average time and  $O(n)$  worst time.

For more information, refer to the article - [unordered\\_multimap in C++](#).

## 47. What is priority\_queue?

A `priority_queue` is a container adapter that is used to create a queue whose order of operation is based on the priority of the element. The higher priority element will come out first instead of the least recent one.

It is implemented using heap data structures in C++.

**Syntax:**

```
priority_queue < object_type > heap_name;
```

By default, this will create a max-heap in which the largest key will pop first. For more information, refer to the article - [Priority\\_queue in C++](#).

## 48. How to create a min-heap using STL priority\_queue?

We can create a min-heap using `priority_queue` by the following method:

**Syntax for Min heap:**

```
priority_queue < object_type , vector<object_type> ,  
greater<object_type> > heap_name;
```

The third argument is the comparator function or functor with a boolean return type. We have used `greater<>` which is a built-in functor of STL for comparing two

values to check which one is greater.

Another way to create a min-heap using `priority_queue` is to just multiply the keys by -1 before inserting them into the queue.

## 49. What are the basic operations on `priority_queue` in C++?

The basic operations that can be performed on the `priority_queue` are as follows:

- **`push( )`**: used for insertion of the element
- **`pop( )`**: used for removing top element
- **`top( )`**: used for checking either min for min-heap and max for max-heap.
- **`size( )`**: used for getting the size of the heap.

We can only access a top element of the heap.

## 50. How is `priority_queue` implemented in C++ STL ? What is the time complexity of basic operations in it?

The `priority_queue` is implemented as a heap data structure in C++. As heap is implemented using an array, the `priority_queue` in STL is also implemented using STL vectors which are nothing but dynamic arrays.

The time complexity of basic operations in `priority_queue` is as follows:

- **`push( )`**:  $O(\log N)$
- **`pop( )`**:  $O(\log N)$
- **`top( )`**:  $O(1)$
- **`size( )`**:  $O(1)$

## Bouns Questions

1. Write code to iterate from the first element to the last element when the vector is given

```
void iter(vector<int> v)
{
```

```

vector<int>::iterator it1,it2;

for(it1=v.begin(),it2=v.end(),it2;it1!=it2;it1++)
{
    cout<<*it1<<" ";
}

cout<<endl;
}

```

## 2. Write code to sort and reverse the vector.

```

vector<int> sortVector(vector<int> v)
{
    sort(v.begin(), v.end());
    return v;
}

vector<int> reverseVector(vector<int> v)
{
    for(int i = 0, j = v.size() - 1; i < j; i++, j--)
        swap(v[i],v[j]);

    return v;
}

```

## 4. Declare, Insert and Print a pair of strings paired with another pair of int paired with int.

```

//Declaration of Pair
pair<string,pair<int,int>> x;

//Inserting element
x= { "Geeks" , { 1, 2} };

```

```
//Printing element
cout<<x.first<<" "<<x.second.first<<" "<<x.second.second;
```

5. Find the index of a given element in a vector, such that the next element to it is 2, and if there exists more than one such index returns the larger one if there is no such index then return -1.

```
int find_index(vector<int> &V,int a)
{
    stack<int> temp;

    for(int i=0;i<V.size();i++)
    {
        if(V[i]==a){
            temp.push(i);
        }

        if(i==temp.top()+1){
            if(V[i]==2)
                continue;
            else if(V[i]==a){
                temp.pop();
                temp.push(i);
            }
            else{
                temp.pop();
            }
        }
    }

    if(temp.top()==V.size()-1)
        temp.pop();

    if(temp.size()==0)
        return -1;
```

```
    return temp.top();  
}
```

## 6. Write a Class stack to implement a stack using a queue.

```
class Stack {  
    queue<int> q1, q2;  
  
public:  
    void push(int x)  
    {  
        q2.push(x);  
  
        while (!q1.empty()) {  
            q2.push(q1.front());  
            q1.pop();  
        }  
  
        queue<int> q = q1;  
        q1 = q2;  
        q2 = q;  
    }  
  
    void pop()  
    {  
        if (q1.empty())  
            return;  
        q1.pop();  
    }  
  
    int top()  
    {  
        if (q1.empty())  
            return -1;  
        return q1.front();  
    }  
  
    int size()
```

```
    {  
        return q1.size();  
    }  
};
```

**7. Write a code to perform push from the back, pop from the front, get the size, get back and get front operations.**

```
void push(queue<int> &q,int x)  
{  
    q.push_back(x);  
}  
  
int pop(queue<int> &q)  
{  
    int x=getFront(q);  
    q.pop_front();  
  
    return x;  
}  
  
int getSize(queue<int> &q)  
{  
    return q.size();  
}  
  
int getBack(queue<int> &q)  
{  
    return q.back();  
}  
  
int getFront(queue<int> &q)  
{  
    return q.front();  
}
```

8. Write a code to check duplicate values in the vector and print them with the best possible time complexity.

```
void duplicate(vector<int> &V)
{
    unordered_set<int> store;

    for(auto it:V)
    {
        if(store.find(it)!=store.end())
        {
            cout<<it<<" ";
        }
        else
        {
            store.insert(it);
        }
    }
}
```

[Comment](#)[More info](#) ▼[Advertise with us](#)[Next Article](#) >

Introduction to C++ Programming  
Language

## Similar Reads

### C++ Programming Language

C++ is a computer programming language developed by Bjarne Stroustrup as an extension of the C language. It is known for its fast speed, low level memory management and is often taught as first...

🕒 5 min read

[C++ Overview](#)[C++ Basics](#)

**C++ Variables and Constants**



**C++ Data Types and Literals**



**C++ Operators**



**C++ Input/Output**



**C++ Control Statements**



**C++ Functions**



**C++ Pointers and References**



Do Not Sell or Share My Personal Information